Федеральное государственное образовательное бюджетное учреждение высшего образования

«Финансовый университет при Правительстве Российской Федерации»

Факультет «Информационных технологий и анализа больших данных»

Направление «Программная инженерия»

Практическая работа №2

По дисциплине **«Кроссплатформенная разработка»**

На тему **«Знакомство с инструментальными средствами разработки кросс-платформенных программных продуктов»**

Выполнила: студентка 4 курса,

группы ТРПО23-1

очной формы обучения

Бурняшева Елена Александровна.

Преподаватель:

Бузыкова Ю.С.

Москва 2026

Задание 1. Классификация инструментальных средств

Дата проверки сведений: 15.10.2026.

В таблице рассматриваются инструменты разных категорий: среды выполнения, виртуальные машины, системы сборки, десктопные и мобильные фреймворки, WebView-обёртки и средства контейнеризации.

Для каждого инструмента указаны назначение, механизм переносимости, лицензия, версия и дата выпуска.

Сведения о релизах проверяются по официальным источникам.

Для инфраструктурных инструментов в графе примера указывается проект, использующий инструмент, или пример его применения: система сборки и контейнерная среда сами по себе не являются фреймворками создания интерфейса.

Классификационная таблица

Категория	Инструмент	Разработчик	Язык программирования	Целевые платформы	Механизм кросс-платформенности	Лицензия	Версия и дата релиза	Пример приложения или использования
Интерпретируемые языки и среды выполнения	Python, реализация CPython	Сообщество Python; Python Software Foundation	Python — язык приложений	Windows, macOS, Linux и другие поддерживаемые платформы	Выполнение общего Python-кода в реализации интерпретатора для целевой платформы; переносимость зависит от используемых библиотек и системных API	Python Software Foundation License Version 2; дополнительные условия для отдельных компонентов	3.15.0 — 09.10.2026	Blender: встроенный Python API для интерфейса, инструментов и расширений; не всё приложение написано на Python
Переносимый исходный код и кросс-платформенная сборка	CMake	Kitware и сообщество	Язык CMake для описания сборки; проекты, в частности, на C и C++	Windows, macOS, Linux и другие платформы; целевые системы зависят от компилятора и toolchain	Общие описания сборки преобразуются в файлы для выбранной системы сборки; исходный код компилируется под целевую платформу	BSD-3-Clause	4.4.4 — 08.10.2026	Blender: CMake используется как система конфигурирования сборки
Виртуализация и контейнеризация	Docker Desktop	Docker, Inc.	Не ограничивает язык приложений внутри контейнеров	Windows, macOS, Linux с соблюдением системных требований	Контейнеризация и подготовленная среда выполнения; для Linux-контейнеров используется Linux-ядро, при необходимости через виртуализированное окружение	Docker Subscription Service Agreement; отдельные компоненты имеют собственные открытые лицензии	4.95.0 — 12.10.2026	WordPress с MySQL: развёртывание и тестирование в контейнерах; Docker используется для окружения, а не как язык приложения
Веб-технологии и WebView-обёртки	Electron	Сообщество Electron; проект под управлением OpenJS Foundation	JavaScript / TypeScript, HTML, CSS; при необходимости нативные модули	Windows, macOS, Linux	Приложение поставляется со встроенными Chromium и Node.js; общий веб-интерфейс выполняется в одинаковом браузерном движке на поддерживаемых ОС	MIT; встроенные сторонние компоненты имеют собственные лицензии	44.7.2 — 13.10.2026	Visual Studio Code
Веб-технологии и WebView-обёртки	Tauri 2	Сообщество Tauri / Tauri Programme в рамках The Commons Conservancy	JavaScript / TypeScript, HTML, CSS для интерфейса; Rust для нативной логики; Kotlin и Swift для мобильных интеграций	Windows, macOS, Linux, Android, iOS	Общий веб-интерфейс отображается в системном WebView; нативная часть компилируется для целевой платформы, взаимодействие осуществляется через API Tauri	MIT OR Apache-2.0 для основных компонентов; зависимости проверяются отдельно	2.12.4 — 14.10.2026, версия Rust-пакета tauri	GitButler — клиент системы контроля версий
Виртуальные машины и байт-код	JVM в составе Oracle JDK	Oracle; развитие Java также ведётся сообществом OpenJDK	Java, Kotlin, Scala и другие языки, компилируемые в JVM-байт-код	Windows, macOS, Linux; доступность зависит от сборки JDK и архитектуры	Исходный код компилируется в JVM-байт-код, который выполняется виртуальной машиной для конкретной платформы	Для выбранной поставки Oracle JDK 27 — Oracle No-Fee Terms and Conditions (NFTC); другие дистрибутивы JDK могут иметь иные лицензии	Oracle JDK 27 — 15.09.2026	IntelliJ IDEA — приложение, выполняющееся на JVM; обычно использует собственную поставку JetBrains Runtime
Десктопные UI-фреймворки	Qt 6	Qt Group и сообщество Qt Project	C++, QML; JavaScript для логики QML	Windows, macOS, Linux; также поддерживаются мобильные, встраиваемые платформы и WebAssembly в	Общий API библиотек и интерфейса скрывает различия платформ; приложение компилируется под целевую ОС, для Qt Quick используется	Коммерческая лицензия либо открытые лицензии LGPLv3/GPLv3; условия	6.12.0 — 30.09.2026	Qt Creator — интегрированная среда разработки на Qt

				рамках соответствующих конфигураций	собственная система рендеринга	зависят от модуля		
Мобильные кросс-платформенные фреймворки	React Native	Meta и сообщество	JavaScript / TypeScript; нативные расширения на языках платформ	Android, iOS; другие платформы поддерживаются отдельными проектами экосистемы	Общий React-код управляет нативными компонентами через инфраструктуру React Native; платформенные различия обрабатываются API, расширениями и условными реализациями	MIT	0.87.1 — 26.08.2026	Shopify и Shopify POS
Веб-технологии и WebView-обёртки	Capacitor	Ionic и сообщество	JavaScript / TypeScript, HTML, CSS; Swift и Java/Kotlin для нативных плагинов	Android, iOS, Web	Веб-приложение работает внутри нативного контейнера на мобильных платформах; доступ к возможностям устройства предоставляется плагинами	MIT	8.5.3 — 07.10.2026	BBC Children's Games: доставка веб-игр на iOS и Android с использованием Capacitor
Интерпретируемые языки и среды выполнения	Node.js	Сообщество Node.js; проект OpenJS Foundation	JavaScript; TypeScript требует подходящего режима выполнения или предварительного преобразования	Windows, macOS, Linux и другие поддерживаемые системы	Общий JavaScript-код выполняется средой Node.js для целевой платформы; движок V8 использует в том числе JIT-компиляцию, а API Node.js абстрагируют многие системные операции	MIT; сторонние компоненты имеют собственные лицензии	26.12.0, ветка Current — 14.10.2026	Инфраструктурный пример: Vite — инструмент разработки и сборки веб-приложений

Различие Electron и Tauri

Electron включает Chromium и Node.js в поставку приложения. Это даёт разработчику единый браузерный движок на поддерживаемых настольных ОС, но увеличивает объём поставляемых компонентов.

Tauri использует системный WebView и нативную часть приложения. Отсутствие встроенного браузерного движка позволяет уменьшить размер поставки, однако требует учитывать особенности и зависимости WebView на каждой целевой платформе.

Точный размер дистрибутива и потребление памяти нельзя определить только по названию фреймворка: для корректного сравнения необходимы сопоставимые приложения, настройки сборки и условия измерения.

Источники

Python

- Релизы Python: https://www.python.org/downloads/
- Лицензия Python: https://docs.python.org/3/license.html
- Использование Python в Blender: https://www.blender.org/about/license/

CMake

- Версия для загрузки: https://cmake.org/download/
- Новости и даты релизов: https://cmake.org/
- Лицензия: https://cmake.org/licensing/
- Сборка Blender: https://developer.blender.org/docs/handbook/building_blender/

Docker Desktop

- Релизы: https://docs.docker.com/desktop/release-notes/
- Описание продукта: https://docs.docker.com/desktop/
- Лицензирование: https://docs.docker.com/subscription-billing/desktop-license/

Electron

- Описание, платформы, лицензия и примеры:

https://www.electronjs.org/docs/latest/why-electron

- Стабильные релизы:

https://releases.electronjs.org/

Tauri

- Поддерживаемые платформы:

https://v2.tauri.app/

- Архитектура и использование системного WebView:

https://v2.tauri.app/start/

- Релизы:

https://github.com/tauri-apps/tauri/releases

- Лицензирование основных компонентов:

https://github.com/tauri-apps/tauri

- Пример приложения GitButler:

https://github.com/gitbutlerapp/gitbutler

JVM / Oracle JDK

- Описание Java-платформы и JVM:

https://docs.oracle.com/javase/8/docs/technotes/guides/index.html

- Анонс выпуска Oracle JDK 27:

https://www.oracle.com/news/announcement/oracle-releases-java-27-and-strengthens-post-quantum-cryptography-support-2026-09-15/

- Доступные платформы и условия распространения Oracle JDK:

https://www.oracle.com/java/technologies/downloads/

- Использование JVM в IntelliJ IDEA:

https://www.jetbrains.com/help/idea/tuning-the-ide.html

Qt

- Анонс Qt 6.12:

https://www.qt.io/blog/qt-6.12-released

- Лицензирование:

https://doc.qt.io/qt-6/licensing.html

- Qt Creator:

https://www.qt.io/development/tools/qt-creator-latest-releases

React Native

- Описание технологии и разработчик:

https://reactnative.dev/

- Поддерживаемые версии:

https://reactnative.dev/versions

- Релизы:

https://github.com/react/react-native/releases

- Репозиторий и лицензия:

https://github.com/react/react-native

- Примеры приложений:

https://reactnative.dev/showcase

Capacitor

- Описание и платформы:

https://capacitorjs.com/

- Репозиторий и лицензия:

https://github.com/ionic-team/capacitor

- Релизы:

https://github.com/ionic-team/capacitor/releases

Node.js

- Описание среды:

https://nodejs.org/

- Релиз 26.12.0:

https://nodejs.org/en/blog/release/v26.12.0

- Репозиторий и лицензия:

https://github.com/nodejs/node

- Пример инструмента Vite:

https://vite.dev/guide/

Дата обращения ко всем перечисленным источникам: 15.10.2026.

Дополнительные источники примеров:

https://www.docker.com/blog/how-to-dockerize-wordpress/

https://ionic.io/resources/case-studies/bbc-games

Дата обращения: 15.10.2026.

Задание 2. Подготовка рабочего окружения

Проверка инструментов

Работа выполнялась на компьютере под управлением Windows 10 с использованием локально установленных инструментов.

В командной строке были выполнены команды:

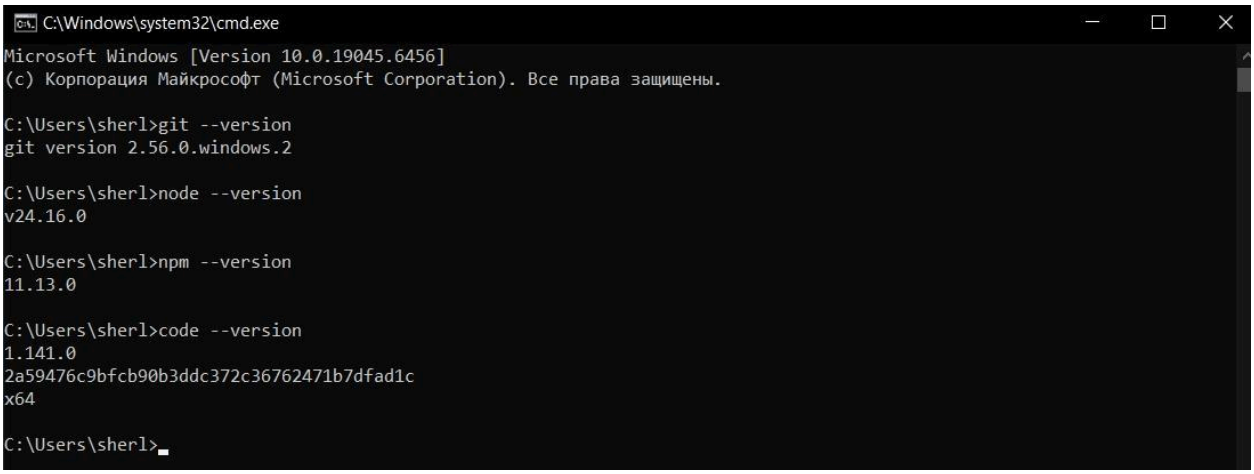
- git --version;
- node --version;
- npm --version;
- code --version.

Получены следующие версии:

Инструмент	**Версия**
Git	2.56.0.windows.2
Node.js	24.16.0
npm	11.13.0
Visual Studio Code	1.141.0, x64

Node.js относится к ветке 24 LTS.

Проверка версий инструментов



Создание и клонирование репозитория

На GitHub создан публичный репозиторий crossplatform-practice-02.

https://github.com/burning-helen/crossplatform-practice-02

Репозиторий клонирован на локальный компьютер командой git clone и открыт в Visual Studio Code.

В README.md указаны тема работы, ФИО обучающегося, группа и индивидуальный вариант.

Фиксация и отправка изменений

После редактирования README.md и добавления скриншота окружения изменения включены в локальный коммит и отправлены на GitHub.

Использованные команды:

git add README.md report/

git commit -m "Add practice description and environment screenshot"

git push

Для просмотра истории использована команда:

git log --oneline -5

История коммитов



Результат

Рабочее окружение подготовлено. Создан удалённый репозиторий и его локальная копия. Выполнены локальный коммит и отправка изменений на GitHub. Подтверждающие скриншоты сохранены для включения в итоговый отчёт.

Задание 3. Исследование структуры кросс-платформенного проекта

Исследуемый проект

Для анализа выбран официальный пример Tauri API из репозитория tauri-apps/tauri, расположенный в каталоге examples/api.

Адрес репозитория: https://github.com/tauri-apps/tauri

Каталог примера: examples/api

Дата обращения: 15.10.2026

Коммит: 152d293ddf5680b968dc4853e52d87c12a2deaeb

Пример демонстрирует возможности API Tauri. Веб-интерфейс реализован с использованием Svelte и TypeScript, а нативная часть — на Rust. Для веб-сборки используется Vite.

Проект исследуется без запуска и сборки.

Пример входит в монорепозиторий Tauri и использует его общие компоненты: локальные зависимости JavaScript и зависимости Rust, наследуемые из Cargo workspace.

Структура проекта

Каталог или файл	**Назначение**	**Общая кодовая база или платформенная часть**
src/	Исходные файлы веб-интерфейса приложения	Общая веб-часть; конкретные обращения к ОС требуют отдельного анализа
src/main.ts	Входной файл веб-части	Общая кодовая база
src/App.svelte	Корневой компонент интерфейса	Общая кодовая база
src/app.css	Стили интерфейса	Общая кодовая база
src/components/	Компоненты интерфейса, включая компоненты работы с меню	Общая веб-часть
src/views/	Представления для демонстрации возможностей приложения: окна, меню, трей, коммуникация и другие	Общая веб-часть; отдельные возможности могут иметь платформенные ограничения
index.html	Входной HTML-документ веб-части	Общая кодовая база
public/	Статические ресурсы веб-части	Общие ресурсы
dist/	Каталог результата сборки веб-части; в исследуемом дереве содержит только .gitkeep	Общий результат веб-сборки, а не отдельная реализация для ОС
isolation-dist/	Файлы isolation-приложения, подключённого в настройках безопасности	Общая инфраструктура безопасности
package.json	Зависимости JavaScript, метаданные примера и команды Vite и Tauri CLI	Общая конфигурация
vite.config.js	Конфигурация сборщика Vite	Общая конфигурация веб-сборки
svelte.config.js	Конфигурация Svelte	Общая конфигурация
tsconfig.json	Конфигурация TypeScript	Общая конфигурация
unocss.config.js	Конфигурация UnoCSS	Общая конфигурация оформления
src-tauri/src/	Исходные файлы нативной части на Rust	Смешанная часть: общий нативный код и возможные платформенные ветвления
src-tauri/src/main.rs	Точка входа исполняемой Rust-части; детали определяются содержимым файла	Нативная часть
src-tauri/src/lib.rs	Библиотечная часть Rust-приложения	Нативная часть; общий и платформозависимый код нужно различать по исходникам
src-tauri/src/cmd.rs	Модуль команд приложения; реализация исследуется отдельно	Нативная часть
src-tauri/src/menu_plugin.rs и tray.rs	Модули, связанные с меню и системным треем	Нативная интеграция; ограничения платформ проверяются по исходникам
src-tauri/Cargo.toml	Метаданные Rust-пакета, зависимости, возможности сборки и платформенное условие подключения зависимости	Общая конфигурация с платформозависимым разделом

src-tauri/build.rs	Скрипт этапа сборки Rust-пакета	Инфраструктура нативной сборки
src-tauri/tauri.conf.json	Настройки приложения, веб-сборки, безопасности и упаковки	Общая конфигурация с платформенными параметрами
src-tauri/Info.plist	Файл метаданных для платформ Apple; конкретные ключи требуют просмотра	Платформенная конфигурация Apple
src-tauri/capabilities/	Конфигурация предоставления разрешений окнам и WebView	Общая инфраструктура безопасности с возможными ограничениями по платформам
src-tauri/permissions/	Описания разрешений для команд приложения	Общая инфраструктура безопасности
src-tauri/locales/pt-BR.wxl	Локализация установщика WiX, подключённая в конфигурации	Windows
src-tauri/tauri-plugin-sample/	Локальный пример плагина, подключённый как Rust-зависимость	Общая оболочка плагина и платформенные реализации
src-tauri/tauri-plugin-sample/src/desktop.rs	Реализация плагина для настольных платформ, подключаемая через #[cfg(desktop)]	Код для категории настольных платформ
src-tauri/tauri-plugin-sample/src/mobile.rs	Реализация плагина для мобильных платформ, подключаемая через #[cfg(mobile)]	Код для категории мобильных платформ
src-tauri/tauri-plugin-sample/android/	Android-часть плагина: Kotlin-код, манифест, настройки Gradle и тесты	Android
src-tauri/tauri-plugin-sample/android/build.gradle.kts	Конфигурация сборки Android-части плагина	Android
src-tauri/tauri-plugin-sample/ios/	iOS-часть плагина: Swift-код и тесты	iOS
src-tauri/tauri-plugin-sample/ios/Package.swift	Манифест Swift-пакета iOS-части плагина	iOS

Особенности конфигурации

1. Веб-часть использует Svelte и TypeScript. Команды dev и build запускают Vite.

2. JavaScript API Tauri подключается по относительному пути

../../packages/api/dist, а команда Tauri CLI использует файл

../../packages/cli/tauri.js. Это указывает на связь примера с остальной частью монорепозитория.

3. Rust-зависимости tauri и tauri-build наследуются из Cargo workspace. Поэтому их конкретные версии нельзя определить только по Cargo.toml исследуемого примера.

4. Каталог ../dist указан как источник собранного веб-интерфейса. Перед запуском и сборкой конфигурация предусматривает команды pnpm dev и pnpm build. Наличие npm на компьютере не означает, что все зависимости этого примера уже подготовлены к сборке.

5. Для Windows в tauri.conf.json заданы параметры установщиков WiX и NSIS. Для WiX подключена локализация pt-BR.

6. Зависимость tauri-plugin-automation объявлена только для целей, отличных от Android и iOS. Дополнительно она является необязательной и включается возможностью automation.

7. Мобильные каталоги android/ и ios/ в исследуемом дереве относятся к вложенному примеру плагина, а не к отдельным корневым проектам приложения.

8. Значения version в package.json, Cargo.toml и tauri.conf.json описывают версии соответствующих компонентов примера. Их нельзя использовать как актуальную версию фреймворка Tauri.

Обращения к платформенным возможностям

1. Управление нативным окном через общий API

Файл: src/views/Window.svelte.

```
function setTitle() {
```

```
  selectedWebview.setTitle(windowTitle)
}
function minimize() {
  selectedWebview.minimize()
  setTimeout(selectedWebview.unminimize, 2000)
}
```

Веб-интерфейс обращается к возможностям нативного окна через API Tauri: изменяет заголовок, сворачивает и восстанавливает окно. Общий код вызывает единые методы, а взаимодействие с конкретной ОС обеспечивает Tauri. Таким образом, обращение к платформенным возможностям не всегда требует отдельной реализации интерфейса для каждой ОС.

2. Условная логика интерфейса в зависимости от ОС

Файл: src/views/Window.svelte.

```
const isWindows = navigator.appVersion.includes('Windows')
const isMacOS = navigator.appVersion.includes('Macintosh')
const effectOptions: Effect[] = isWindows
  ? windowsEffects
  : (Object.values(Effect) as Effect[]).filter(
      (e) => !windowsEffects.includes(e)
    )
```

В разметке блок настройки эффектов отображается условно:

```
{#if isWindows || isMacOS}
```

Пример проверяет строку navigator.appVersion и в зависимости от результата выбирает набор эффектов. Панель эффектов отображается только при распознавании Windows или macOS. Это платформенное ветвление внутри общей веб-кодовой базы. Наличие общего файла не означает, что все его возможности одинаково доступны на каждой платформе. При анализе важно также учитывать, что проверка строки браузерного окружения не равнозначна проверке поддержки конкретного эффекта.

3. Выбор реализации плагина при компиляции

Файл: src-tauri/tauri-plugin-sample/src/lib.rs.

```
#[cfg(desktop)]
mod desktop;
#[cfg(mobile)]
mod mobile;
#[cfg(desktop)]
use desktop::Sample;
#[cfg(mobile)]
use mobile::Sample;
```

В зависимости от целевой категории платформы подключается модуль desktop или mobile. Условие обрабатывается при компиляции, а не во время выполнения приложения. Общая часть плагина использует тип Sample, но его реализация выбирается для соответствующей категории платформ. Аналогичное разделение применяется при инициализации плагина.

4. Возможность, реализованная только на мобильных платформах

Файл: src-tauri/tauri-plugin-sample/src/lib.rs.

```
#[cfg(mobile)]
return app.sample().js_values().map_err(|e| e.to_string());


#[cfg(desktop)]
{
  let _ = app;
  Err("js_values is only implemented on mobile".into())
}
```

Команда js_values имеет различное поведение в зависимости от целевой платформы. На мобильной платформе вызывается реализация метода, а на настольной возвращается ошибка об отсутствии реализации. Этот фрагмент показывает ограничение переносимости: команда зарегистрирована в общем интерфейсе плагина, но соответствующая функциональность доступна не на всех платформах.

Вывод

Системные требования Tauri:

https://v2.tauri.app/start/prerequisites/

Дата обращения: 15.10.2026.

Исследованный пример Tauri API использует веб-технологии для интерфейса и Rust для нативной части приложения. По классификации инструментов он относится к категории веб-технологий и WebView-обёрток.

Переносимость обеспечивается общим веб-интерфейсом, API Tauri и сборкой нативной части под целевую платформу.

Общая веб-кодовая база размещена преимущественно в каталоге src. Каталог src-tauri содержит нативный код, настройки и компоненты, среди которых необходимо различать общие и платформозависимые части. Поэтому считать весь Rust-код платформозависимым некорректно. В компоненте Window.svelte обнаружены обращения к нативным окнам через единый API, а также условный выбор возможностей в зависимости от распознанной ОС.

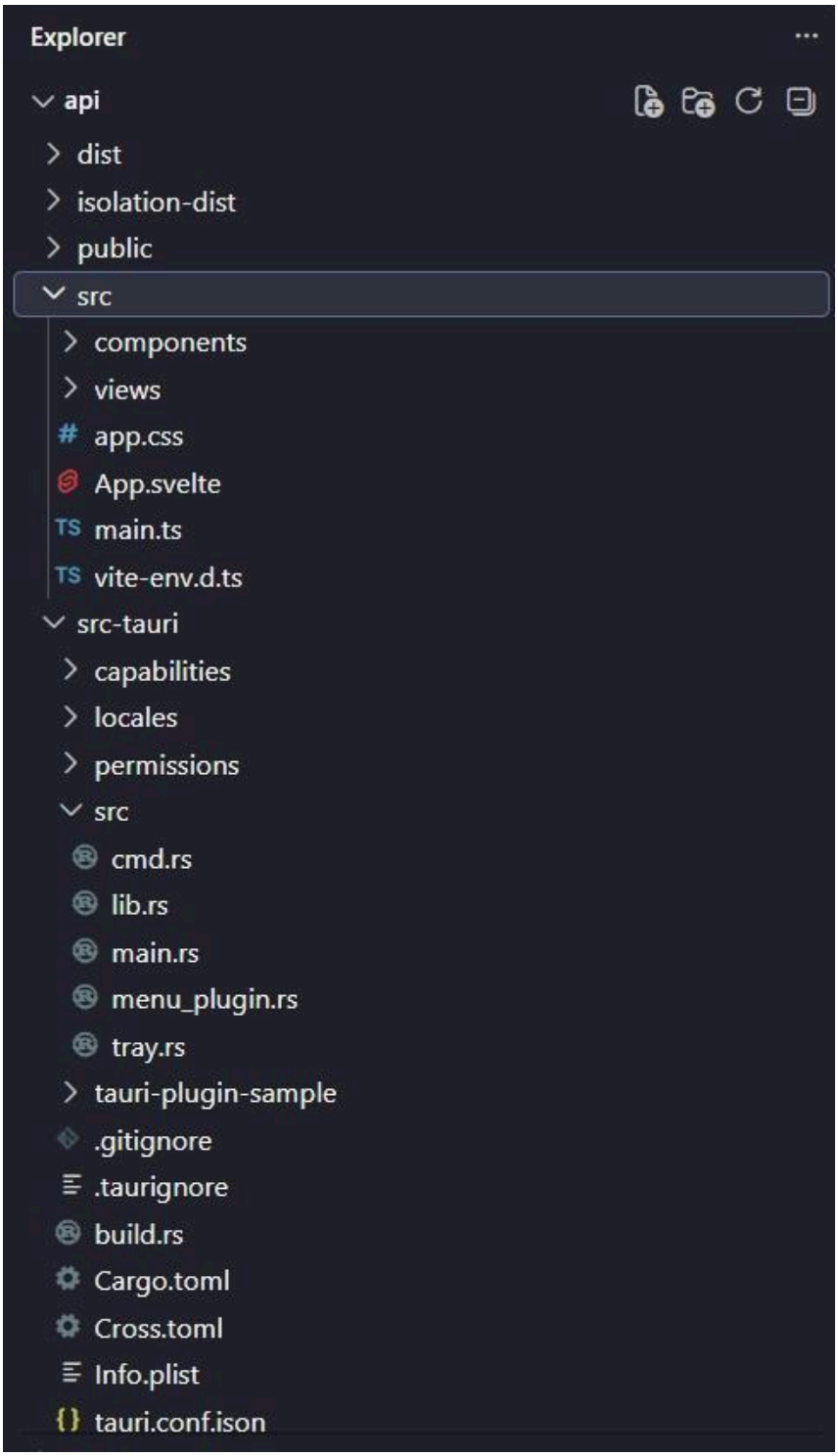
В примере плагина разделение настольной и мобильной реализаций осуществляется при компиляции через атрибуты cfg. Команда js_values демонстрирует функциональное ограничение: на настольных платформах она возвращает сообщение об отсутствии реализации. Количественная доля общего кода не измерялась, однако структура показывает повторное использование веб-компонентов и общей оболочки плагина при наличии выделенных платформенных реализаций.

При переносе на дополнительную платформу требуется проверить поддержку Tauri, системные зависимости, работу оконных функций и реализаций плагинов. Для Astra Linux необходимо отдельно проверить совместимость конкретной версии дистрибутива с зависимостями Tauri, включая WebKitGTK, и выполнить тестирование приложения в целевой среде.

Перенос на ОС «Аврора» нельзя считать автоматически обеспеченным наличием мобильной реализации плагина: для него необходим отдельный анализ инструментальной поддержки и платформенных интеграций.

Скриншоты исследования

Структура проекта Tauri API



Проверка операционной системы в Window.svelte

```
const isWindows = navigator.appVersion.includes('Windows')
const isMacOS = navigator.appVersion.includes('Macintosh')
const effectOptions: Effect[] = isWindows
  ? windowsEffects
  : (Object.values(Effect) as Effect[]).filter(
      (e) => !windowsEffects.includes(e)
    )
const effectStateOptions = Object.values(EffectState) as EffectState[]
```

Разделение настольной и мобильной реализаций плагина

```
#[cfg(desktop)]
mod desktop;
#[cfg(mobile)]
mod mobile;

mod error;
mod models;

#[cfg(desktop)]
use desktop::Sample;
#[cfg(mobile)]
use mobile::Sample;

pub use error::*;
```

Задание 4. Сравнительный анализ инструментов

Индивидуальный вариант

Вариант № 4.

Сравниваемые инструменты: Electron и Tauri 2.

Сценарий: разработка десктопного клиента корпоративного мессенджера

для Windows, macOS и Linux, включая Astra Linux.

Основные требования: небольшой размер дистрибутива и безопасность.

Дата проверки сведений: 15.10.2026.

Сравнение выполнено по официальной документации без сборки и инструментального тестирования приложений. Поэтому оценки размера, потребления ресурсов и производительности являются качественными, а не результатами измерений.

Сравнительная таблица

Критерий	**Electron**	**Tauri 2**
Основные целевые платформы	Windows, macOS, Linux	Windows, macOS, Linux; также Android и iOS, которые не требуются в данном сценарии
Astra Linux и РЕД ОС	Возможность запуска зависит от версии дистрибутива, архитектуры и совместимости системных библиотек с выбранной версией Electron; необходима проверка в целевой среде	Возможность запуска зависит от версии дистрибутива, архитектуры и наличия совместимых системных библиотек, включая WebKitGTK; необходима проверка в целевой среде
ОС «Аврора»	Не входит в основные официальные целевые платформы Electron; перенос требует отдельного исследования	Не следует приравнивать поддержку Android/iOS к поддержке «Авроры»; перенос требует отдельного исследования
Языки и порог входа	JavaScript/TypeScript, HTML, CSS; удобен веб-команде, но требует понимания разделения main, preload и renderer	Веб-интерфейс на JavaScript/TypeScript, HTML и CSS; для собственной нативной логики требуется Rust
Механизм отображения интерфейса	Встроенный Chromium; общий браузерный движок поставляется вместе с приложением	Системный WebView: WebView2 на Windows, WKWebView на macOS и WebKitGTK на Linux
Ожидаемая производительность	Подходит для сложного веб-интерфейса; результат зависит от организации процессов, обмена сообщениями и реализации приложения	Подходит для веб-интерфейса с нативной логикой на Rust; результат зависит от WebView, IPC и реализации приложения
Размер дистрибутива	Включение Chromium и Node.js увеличивает базовый объём поставки	Отсутствие встроенного Chromium позволяет уменьшить базовый объём поставки; размер установщика зависит также от способа доставки WebView и других зависимостей
Потребление ресурсов	Встроенный браузер и процессы приложения создают накладные расходы; точное потребление памяти требует измерения	Возможен меньший объём накладных расходов, но системный WebView также потребляет память; численное преимущество без измерений не установлено
Безопасность веб-интерфейса	Необходимо соблюдать рекомендации Electron: изоляция контекста, песочница, ограниченный preload API, проверка IPC и отсутствие Node.js-интеграции для недоверенного содержимого	Capabilities и permissions позволяют ограничивать доступ веб-интерфейса к нативным командам; также необходимы CSP, проверка входных данных и ограничение областей доступа
Обновления безопасности	Разработчик должен своевременно обновлять Electron вместе с поставляемыми Chromium и Node.js	Нужно обновлять Tauri, Rust- и JavaScript-зависимости, а также учитывать обновление системного WebView

Экосистема и документация	Зрелая экосистема JavaScript, подробная документация, известные десктопные продукты; совместимость npm-пакетов с конкретным процессом приложения требует проверки	Официальная документация, система плагинов и интеграция с веб-фреймворками; наличие нужных нативных интеграций следует проверять заранее
Релиз, рассматриваемый в работе	Electron 44.7.2 — 13.10.2026	Tauri 2.12.4 — 14.10.2026; Rust-пакет tauri
Динамика развития	Регулярные выпуски, связанные с обновлениями Chromium; поддерживается ограниченное число последних стабильных основных веток	Регулярные выпуски ядра, CLI и плагинов; эти компоненты могут иметь разные номера версий
Лицензия	MIT для Electron; отдельные условия у сторонних компонентов	MIT OR Apache-2.0 для основных компонентов; отдельные условия у зависимостей
Стоимость владения	Нет обязательной платы за лицензию фреймворка; расходы связаны с разработкой, обновлениями, тестированием, подписью и распространением	Нет обязательной платы за лицензию фреймворка; дополнительно следует учитывать освоение Rust и тестирование разных системных WebView
Распространение на Windows	Установщики и корпоративные каналы доставки; возможна публикация в Microsoft Store при соблюдении требований	MSI через WiX или EXE через NSIS; возможна публикация через Microsoft Store при соблюдении требований
Распространение на macOS	Прямая поставка приложения и Mac App Store при соблюдении требований Apple	Прямая поставка приложения и App Store; необходимы соответствующая упаковка и подпись
Распространение на Linux	Пакеты и другие форматы, подготовленные средствами упаковки приложения	Поддерживаются форматы распространения, включая DEB, RPM и AppImage; зависимости и совместимость проверяются для целевого дистрибутива
Google Play и RuStore	Не являются каналами распространения рассматриваемого настольного клиента; Electron сам по себе не создаёт Android-приложение	Для Android-версии доступны мобильная сборка и дальнейшая публикация при соблюдении требований магазина; для текущего десктопного сценария эти каналы не требуются
Предварительная оценка для сценария	Альтернатива при приоритете единого браузерного движка и минимизации потребности в Rust	Предпочтителен при приоритете небольшого дистрибутива, управляемого доступа к нативным API и подтверждённой совместимости с целевыми ОС

Обоснование выбора

Для разработки корпоративного мессенджера в заданном сценарии я выбираю Tauri 2. Главными критериями являются небольшой размер дистрибутива и возможность ограничить доступ веб-интерфейса к нативным функциям приложения.

Первый аргумент — архитектура поставки. Electron включает Chromium и Node.js, тогда как Tauri использует системный WebView [1, 2]. Это позволяет Tauri уменьшить объём компонентов, распространяемых вместе с приложением. Для корпоративного развёртывания такой подход может сократить объём загрузок при установке и обновлении клиентов. Однако точный выигрыш необходимо измерять на сопоставимых сборках, учитывая доставку WebView и системных зависимостей.

Второй аргумент — система capabilities и permissions [3]. Она позволяет предоставлять окнам только необходимые разрешения. Для мессенджера целесообразно отдельно ограничить работу с файлами, открытие внешних ссылок и вызовы нативных команд. Это не гарантирует безопасность автоматически: разработчик должен проверять входные данные, ограничивать IPC и своевременно обновлять зависимости. Защита переписки также требует отдельного проектирования протокола, аутентификации и хранения секретов.

Третий аргумент — наличие средств распространения для Windows, macOS и Linux [4]. Для корпоративного клиента можно подготовить установщики Windows и пакеты Linux, а затем организовать контролируемое развёртывание.

Основное ограничение выбора связано с Astra Linux. До утверждения технологии необходимо проверить конкретный выпуск ОС, архитектуру, доступность WebKitGTK и других библиотек [5]. Дополнительные затраты связаны с освоением Rust и тестированием различий между системными WebView. Electron остаётся обоснованной альтернативой, если важнее единый браузерный движок или готовые интеграции команды [1]. Его также можно использовать безопасно при соблюдении официальных рекомендаций [6]. Следовательно, выбор Tauri является предварительным и должен быть подтверждён прототипом на всех целевых ОС.

Источники

[1] Electron. Why Electron.

https://www.electronjs.org/docs/latest/why-electron

[2] Tauri. Webview Versions.

https://v2.tauri.app/reference/webview-versions/

[3] Tauri. Capabilities.

https://v2.tauri.app/security/capabilities/

[4] Tauri. Distribute.

https://v2.tauri.app/distribute/

[5] Tauri. Prerequisites.

https://v2.tauri.app/start/prerequisites/

[6] Electron. Security.

https://www.electronjs.org/docs/latest/tutorial/security

Дополнительные источники:

- Tauri. Windows Installer.

https://v2.tauri.app/distribute/windows-installer/

- Tauri. App Store.

https://v2.tauri.app/distribute/app-store/

- Electron. Release Timelines.

https://www.electronjs.org/docs/latest/tutorial/electron-timelines

- Electron. Releases.

https://releases.electronjs.org/

- Tauri. Releases.

https://github.com/tauri-apps/tauri/releases

Дата обращения ко всем источникам: 15.10.2026.

Ответы на контрольные вопросы

1. Что такое кросс-платформенное ПО? Какие существуют уровни совместимости?

Кросс-платформенное программное обеспечение способно работать более чем на одной программной или аппаратной платформе.

Различают следующие уровни совместимости:

- Машинный код: выполнение программы обеспечивается совместимой средой, эмуляцией или двоичной трансляцией.
- Байт-код: промежуточный код выполняется виртуальной машиной или соответствующей средой исполнения.
- Исходный код: общие исходники компилируются отдельно для целевых платформ.
- API: приложение использует общий интерфейс библиотек или фреймворка, скрывающий различия ОС.

Эти механизмы могут сочетаться в одном приложении.

2. Чем байт-код JVM отличается от WebAssembly?

JVM-байт-код предназначен для виртуальной машины Java и используется языками Java, Kotlin, Scala и другими.

WebAssembly — переносимый двоичный формат инструкций, который выполняется в браузерах и внебраузерных средах.

Их модели исполнения, наборы инструкций и доступные интерфейсы различаются: JVM предоставляет инфраструктуру Java-платформы, а модуль WebAssembly взаимодействует с окружением через предоставленные ему импорты.

Ни один из этих форматов сам по себе не гарантирует наличие всех необходимых платформенных API.

3. Как устроен рендеринг Flutter и почему ему не нужны нативные компоненты ОС?

Flutter строит интерфейс из виджетов, рассчитывает расположение элементов и формирует изображение с помощью собственного графического движка.

Поэтому обычные элементы интерфейса Flutter не требуют создания соответствующих системных кнопок или текстовых полей. Например, Impeller используется для рендеринга на поддерживаемых мобильных конфигурациях.

При необходимости Flutter также позволяет встроить нативный компонент через механизм platform views.

4. Что изменилось в React Native с переходом на новую архитектуру?

Новая архитектура включает рендерер Fabric, систему нативных модулей TurboModules и интерфейс JSI. Она устраняет зависимость от прежней модели асинхронного сериализованного моста и улучшает взаимодействие JavaScript с нативной частью.

В React Native 0.76 новая архитектура стала включённой по умолчанию, а начиная с 0.82 — единственной доступной.

Поэтому старые описания взаимодействия через bridge нельзя автоматически применять к современным версиям.

5. В чём разница между Kotlin Multiplatform и Compose Multiplatform? Что изменилось в CMP 1.8?

Kotlin Multiplatform позволяет разделять код между платформами, например бизнес-логику, сетевое взаимодействие и модели данных, сохраняя при необходимости отдельные нативные интерфейсы.

Compose Multiplatform — построенный на этой основе фреймворк для разделения кода пользовательского интерфейса.

В версии 1.8.0 поддержка Compose для iOS получила стабильный статус, а основные API — более сильные гарантии совместимости. При этом необходимость платформенных интеграций и тестирования не исчезла.

6. Почему у Electron большой дистрибутив и как эту проблему решает Tauri?

Electron включает Chromium и Node.js в состав приложения, поэтому вместе с прикладным кодом поставляется значительный объём инфраструктуры.

Tauri использует системный WebView и не требует включать собственную копию Chromium в обычную поставку. Это позволяет уменьшить базовый размер приложения, но итоговый размер зависит от ресурсов, зависимостей и способа доставки WebView.

Точное преимущество определяется измерением сопоставимых сборок.

7. Какие инструменты подходят для разработки под ОС «Аврора» и почему?

Основной нативный вариант — официальный SDK «Аврора», C++, Qt и QML. Он предоставляет инструменты и API, адаптированные к этой ОС.

При выборе конкретной версии Qt и библиотек необходимо проверять документацию соответствующего выпуска SDK.

Поддержка Android или Linux у стороннего фреймворка сама по себе не подтверждает его поддержку «Авроры».

8. Чем публикация в RuStore отличается от Google Play?

RuStore и Google Play используют отдельные кабинеты разработчика, правила проверки, механизмы подписи и сервисы магазина.

RuStore поддерживает загрузку APK и AAB; для AAB предусмотрена процедура передачи необходимых материалов подписи, формирования APK и их подписания магазином.

Если приложение использует покупки, push-уведомления или другие сервисы магазина, соответствующие интеграции необходимо проверить или адаптировать.

Кроме Android-приложений, RuStore имеет отдельное направление публикации приложений для ОС «Аврора».

9. Когда нативная разработка предпочтительнее кросс-платформенной? Приведите два примера.

Нативная разработка может быть предпочтительнее, когда основные требования связаны с платформенными возможностями, а создание и сопровождение межплатформенных интеграций не даёт достаточной выгоды.

Примеры:

1. Приложение для специализированного оборудования с SDK только для одной ОС: нативный стек может упростить доступ к устройству и поддержку интеграции.

2. Приложение, которому критически важны новые платформенные API или сложный нативный интерфейс: прямая работа с SDK позволяет не ждать поддержки со стороны фреймворка.

Это не означает, что такие приложения невозможно сделать кросс-платформенными: решение зависит от доступных интеграций и стоимости сопровождения.

10. Какую роль играют CMake и POSIX?

CMake позволяет описать сборку проекта и настроить её для разных компиляторов и платформ.

POSIX стандартизирует системные интерфейсы, использование которых повышает переносимость исходного кода между совместимыми системами.

CMake отвечает за организацию сборки, а POSIX — за согласованность части системных API. Ни один из них автоматически не делает переносимым код с произвольными платформозависимыми вызовами.

11. Почему Cordova не рекомендуют для новых проектов и что пришло на смену?

В контексте методички для новых гибридных приложений предлагается рассматривать Capacitor как более современную альтернативу Cordova.

Capacitor предоставляет нативные проекты и плагинный доступ к возможностям устройства; документация содержит официальный маршрут миграции с Cordova.

При этом некорректно утверждать, что Cordova полностью прекращена или непригодна для всех проектов: выбор зависит от состояния нужных плагинов, требований и существующей кодовой базы.

При миграции необходимо проверить совместимость плагинов и протестировать приложение.

12. Как ИИ-ассистенты влияют на выбор инструментов? Каковы плюсы и минусы в учебном проекте?

ИИ-ассистенты могут ускорять подготовку шаблонного кода, документации и первоначальное знакомство с новым стеком. Однако удобство генерации кода не должно подменять оценку поддерживаемых платформ, архитектуры, лицензии и требований проекта.

Плюсы в учебном проекте:

- ускорение рутинных операций;
- помощь в объяснении кода и ошибок;
- подготовка примеров и вариантов решения.

Минусы:

- возможные ошибки и устаревшие API;
- риск использования кода без понимания;
- необходимость проверки безопасности и зависимостей;
- риск передачи конфиденциальных данных.

Поэтому сгенерированные решения нужно сверять с документацией и проверять самостоятельно.

Источники

1. Методические указания к практической работе № 2.

2. Java SE Platform Overview.

https://docs.oracle.com/javase/8/docs/technotes/guides/index.html

3. Flutter architectural overview.

https://docs.flutter.dev/resources/architectural-overview

4. Flutter FAQ.

https://docs.flutter.dev/resources/faq

5. React Native 0.82 — A New Era.

https://reactnative.dev/blog/2025/10/08/react-native-0.82

6. Compose Multiplatform 1.8.0 Released.

https://blog.jetbrains.com/kotlin/2025/05/compose-multiplatform-1-8-0-released-compose-multiplatform-for-ios-is-stable-and-production-ready/

7. Electron. Why Electron.

https://www.electronjs.org/docs/latest/why-electron

8. Tauri. Webview Versions.

https://v2.tauri.app/reference/webview-versions/

9. Портал разработчиков ОС «Аврора».

https://developer.auroraos.ru/

10. RuStore. Публикация приложений.

https://www.rustore.ru/help/developers/publishing-and-verifying-apps/app-publication

11. RuStore. Загрузка AAB.

https://www.rustore.ru/help/developers/publishing-and-verifying-apps/app-publication/new-version-app/upload-aab

12. Capacitor. Migrating Strategy.

https://capacitorjs.com/docs/cordova/migration-strategy

Дата обращения к интернет-источникам: 15.10.2026.

Общий вывод

В ходе работы рассмотрены категории инструментов кросс-платформенной разработки, проверено локальное окружение и организовано хранение материалов в GitHub. На примере Tauri API исследованы общая веб-часть, нативная часть и платформенные реализации плагина. Сравнение Electron и Tauri показало, что выбор технологии зависит не только от языка команды, но и от требований к поставке, безопасности и поддержке целевых систем. Для заданного корпоративного мессенджера предварительно выбран Tauri при условии проверки зависимостей и работоспособности на конкретном выпуске Astra Linux.